RELAY-EMBEDDED FAULT CLASSIFICATION IN POWER DISTRIBUTION NETWORKS USING DEEP LEARNING METHODS

Kavishka (E/21/143)
Shakkya (E/21/145)
Induka (E/21/327)

Supervisor – Prof. Janaka Ekanayake

Course: EE405 Date: 02 July 2026

Problem Statement & Motivation

Problem Statement

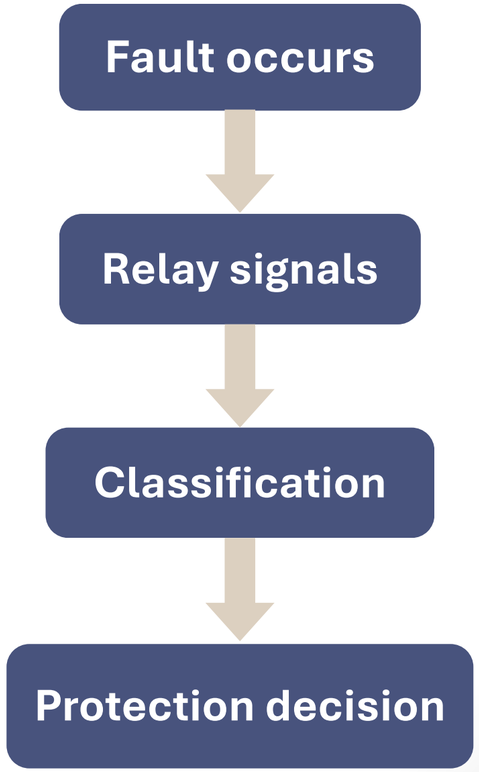
Conventional relay protection may struggle to accurately classify faults under changing operating conditions.

Key Challenges

Fault resistance	Fault location
Fault type	Load variation

Why It Matters

Incorrect or delayed fault classification can reduce grid reliability, safety, and restoration speed.



Literature Review

Approach	Key Idea	Limitation
Conventional Relay Protection [1]	Uses threshold-based current/ voltage settings	Less adaptive under varying fault conditions
Signal Processing Methods [2]	Extract features from fault waveforms	Depends heavily on feature selection
Machine Learning Methods [3]	Classifies faults using learned patterns	Needs suitable training data
Deep Learning Approach [4]	Learns fault patterns directly from relay signals	Requires validation for embedded use

References: [1] Blackburn & Domin, 2014; [2] Li et al., IEEE Trans. Instrum. Meas., 2016; [3] Wang & Chen, 2019; [4] Alhanaf, Balik & Farsadi, 2023.

Objectives & Deliverables

Objectives

- Classify fault types using relay-measurable signals
- Simulate faults under varying operating conditions
- Train and validate a deep learning classifier
- Evaluate accuracy and embedded feasibility

Deliverables

- Simulink-based fault simulation model
- Processed fault signal dataset
- Trained fault classification model
- Validation results and feasibility summary

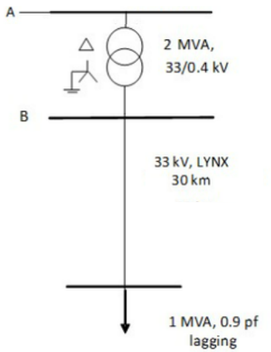
Methodology & Scientific Approach

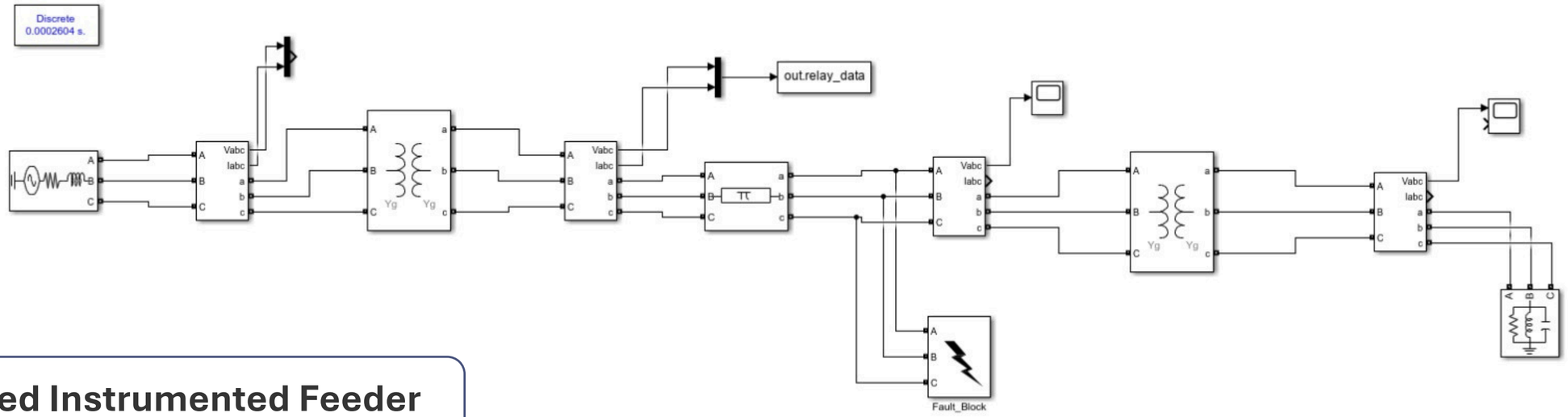
Scientific Approach

Simulink-based fault simulation
Measures voltage/current signals
Signal preprocessing for DL input
Fault type classification using a trained DL model

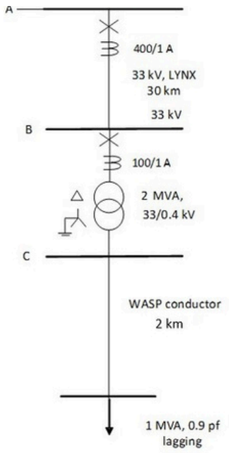
Network Model Development

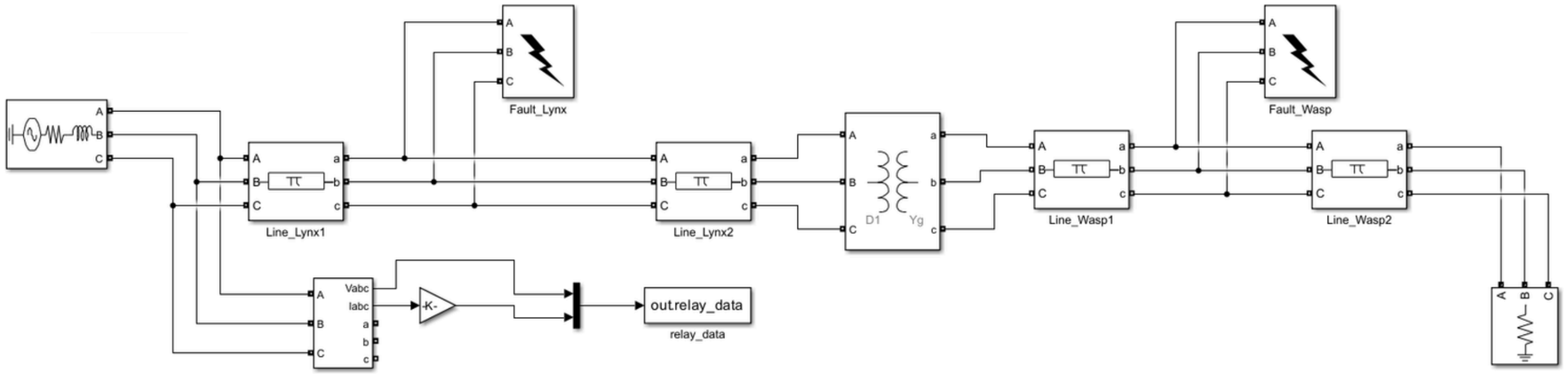
Model 01: Initial Test Feeder





Model 02: Improved Instrumented Feeder





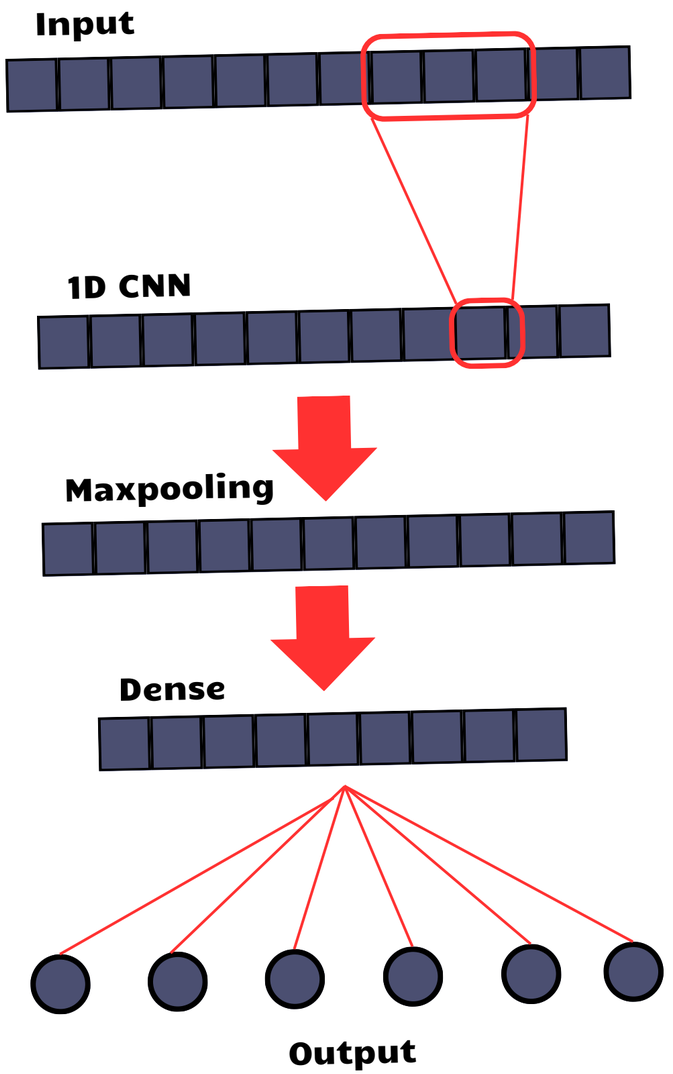
1D-CNN Fault Classification Model

Model Focus

Learns fault patterns
Predicts fault type

Accuracy

Model 01 | 3000 data
99%
Model 02 | 6000 data
14%



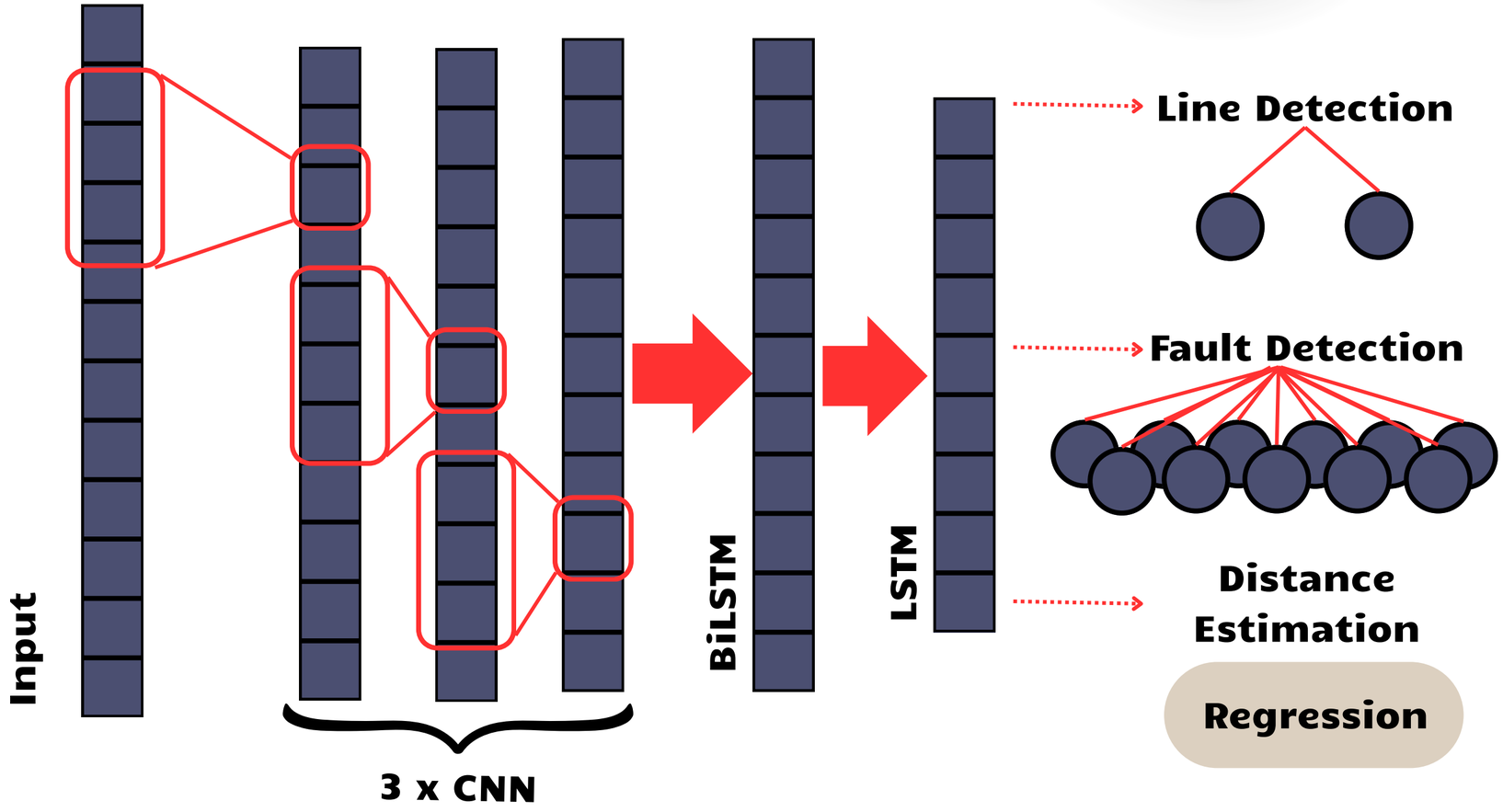
Hybrid CNN-BiLSTM Fault Diagnosis Model

Model Focus

CNN-based feature extraction
BiLSTM/LSTM temporal learning
Fault and line classification
Distance estimation by regression

Accuracy

Model 02 | 6000 data
62.45% to **82.00%**
Model 02 | 8600 data
85.40% to **99.59%**



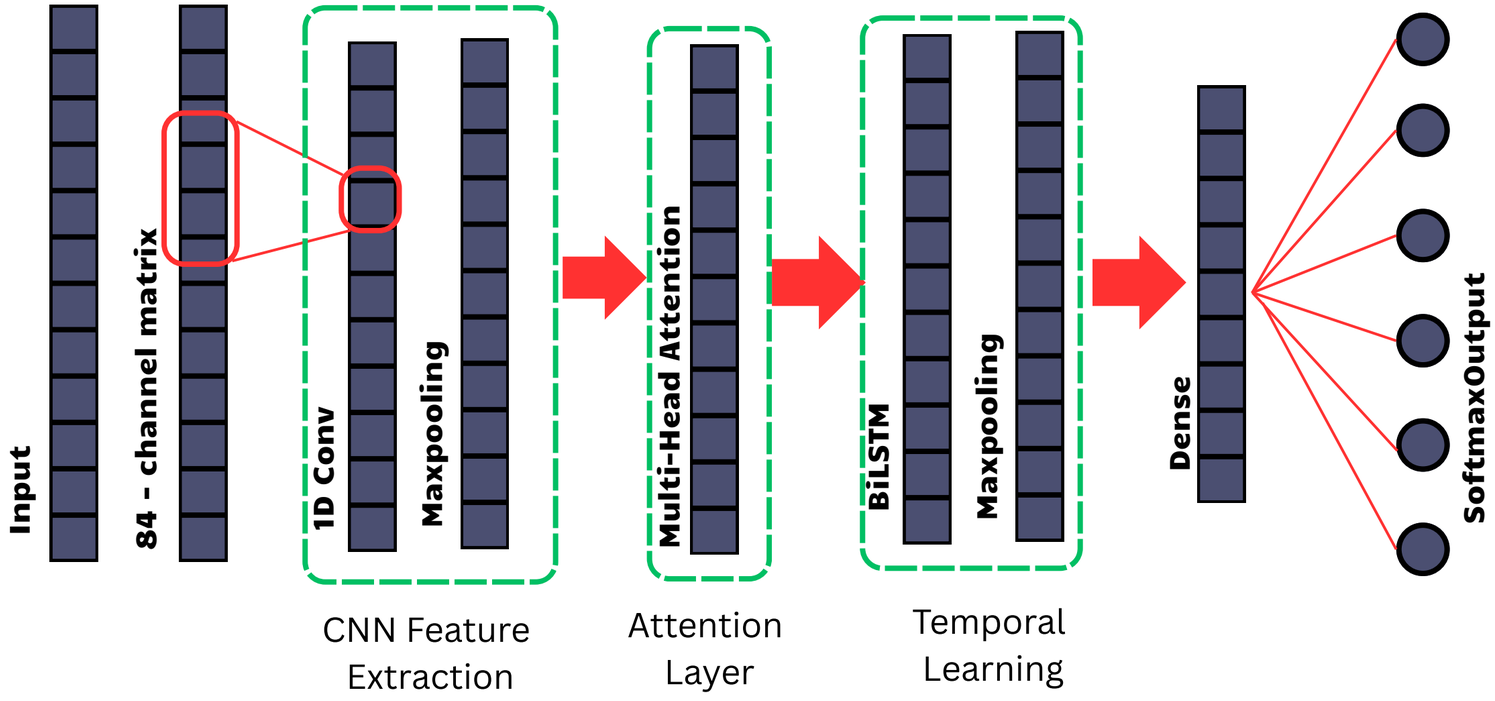
Hybrid CNN-BiLSTM Attention Fault Diagnosis Model

Model Focus

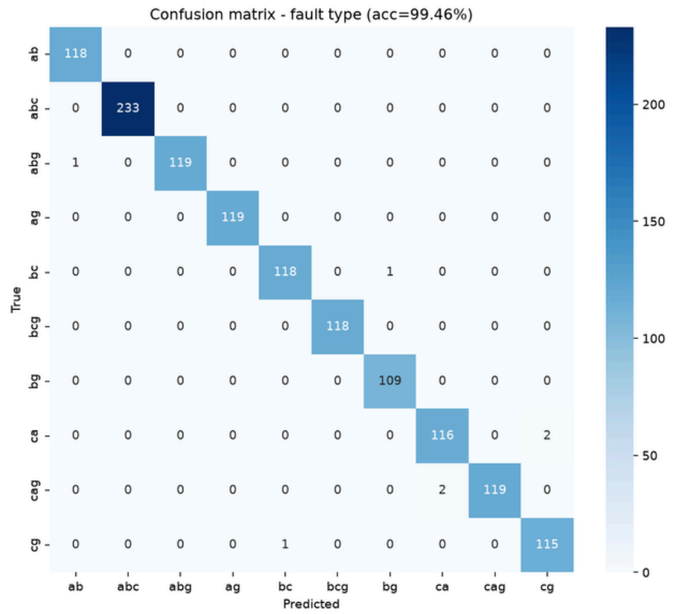
84-channel engineered signal input
Inception-CNN feature extraction
Attention + BiLSTM temporal learning
Softmax-based fault classification

Accuracy

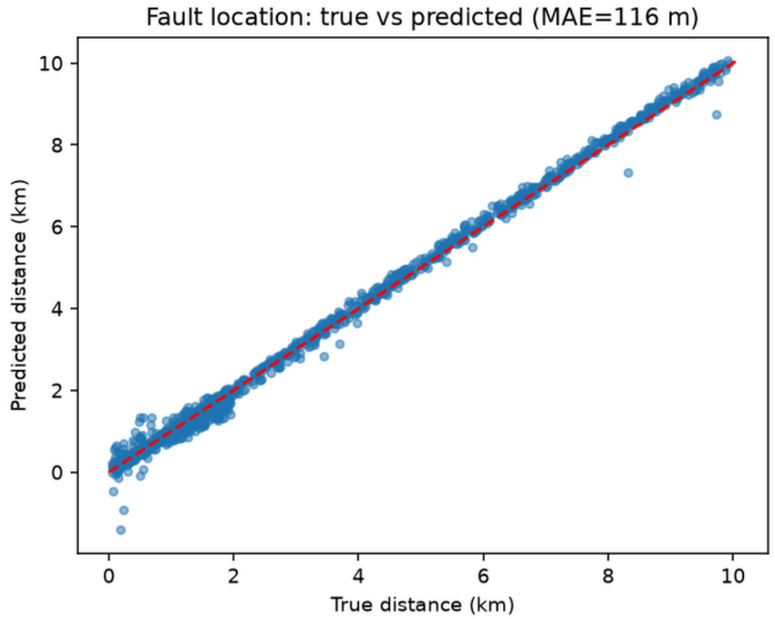
Model 01 | 3000 data
100%
Model 02 | 6000 data
99.33%



Final Results & Observations



Confusion Matrix



Actual vs. Predicted Fault Distance

Fault Distance Estimation :

- Average Error (MAE): The model's distance guess is off by only 116.2 meters on average across the simulated network.
- Trend Match: 0.9967, meaning the AI's predicted distance matches the actual fault distance almost perfectly.

Categorical Performance - Fault Type

- Overall Accuracy was 99.59% across all 10 standard fault types (SLG, LL, LLG, LLL, LLLG).
- Key Finding: Using physical electrical features (like frequency and impedance) fixed the model's confusion between similar faults (like abc and abcg). False alarms during normal grid events dropped to almost zero.

Hardware Readiness:

- Separating the tasks allowed us to remove unneeded parts of the neural network, making the overall model much smaller.
- Next Step: The model is now accurate enough and small enough to be compressed for our final goal: testing it on a physical microcontroller to see if it trips within 10 to 20 milliseconds.

Project Timeline & Progress

Relay-Embedded Fault Classification in Power Distribution Networks Using Deep Learning Methods

Project Timeline

Weeks	Phase / Task	Description	E/21/143	E/21/145	E/21/327
1 – 3	**Literature Review & Tool Familiarisation**	Survey on Fault Classification. Survey ML/DL advances for waveform feature extraction. Explore simulation environments and verify time-series data export pipelines.	Refer Fault Classification papers published before. Set up simulation interface. Test tool compatibility and environment configuration.	Refer Fault Classification papers published before. Review ML/DL architectures suitable for fault classification.	Refer Fault Classification papers published before. Investigate data export methods. Confirm time-series extraction reliability.
4 – 6	**Independent Network Modelling**	Build three separate, fully functional test feeder network models. Each includes distributed generation sources (synchronous and inverter-based).	Build Network Model 1.	Build Network Model 2.	Build Network Model 3.
6 – 7	**Fault Injection & Dataset Generation**	Inject thousands of faults (3-phase, line-to-line, line-to-earth) at varied positions across all three models. Extract raw voltage and current waveform time-series.	Develop and run automation scripts for systematic fault injection.	Define and control extraction parameters to ensure data consistency.	Aggregate and organise the full combined dataset from all three models.
Mid-Semester					
9 – 10	**Data Preprocessing**	Clean and structure the datasets. Apply mathematical transforms to create training/testing splits.	Format and structure data for compatibility with ML pipelines.	Lead advanced preprocessing. Apply signal transforms (e.g. FFT, DWT).	Implement conventional relay logic models as baseline benchmarks.
	Conventional Logic	Implement traditional relay protection algorithms for baseline comparison.			
11 – 12	**ML/DL Model Training & Testing**	Feed preprocessed fault data into the selected ML/DL model. Train, validate, and refine performance across thousands of fault scenarios.	Evaluate model metrics. Assist with hyperparameter tuning.	Lead model training and validation. Drive architecture decisions.	Evaluate model metrics. Assist with hyperparameter tuning.

Conclusions & Next Steps

Key Conclusions

- Raw Data is Not Enough: Basic AI models fail on complex grid faults. Giving the AI physical electrical features (like frequency and impedance) is required for high accuracy.
- Data Pipelines Matter: Moving away from heavy text files (CSV) to fast binary formats (Parquet) solved our major speed and memory bottlenecks.
- Task Separation Works: Splitting the network into two parts—one for guessing the fault type and one for guessing the distance—stopped the AI from getting confused and pushed accuracy to 99.59%.

Next Steps

- Complex Grid Testing: Test our 99.59% model on a larger, more complicated power network (the IEEE 39-bus system) to make sure it still works perfectly.
- Model Compression: Shrink the AI's file size (quantization) so it can fit on a small microcontroller's memory.
- Physical Hardware Testing: Connect the microcontroller to our simulation and prove it can issue a trip signal within the strict 10 to 20-millisecond safety deadline.

THANK YOU

FOR YOUR TIME AND SUPPORT


Q&A

Why use Deep Learning over traditional relays?

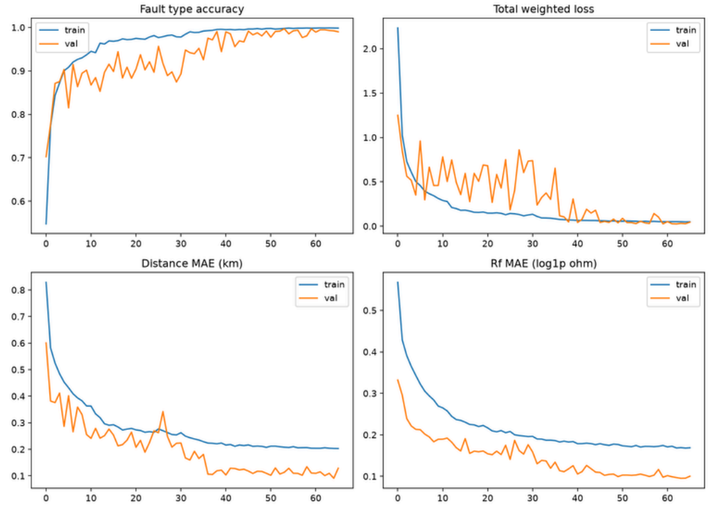
- Answer: Traditional relays use rigid thresholds and fail during complex transients in modern grids. Deep Learning captures the actual "shape" of faults, solving protection gaps traditional relays cannot.

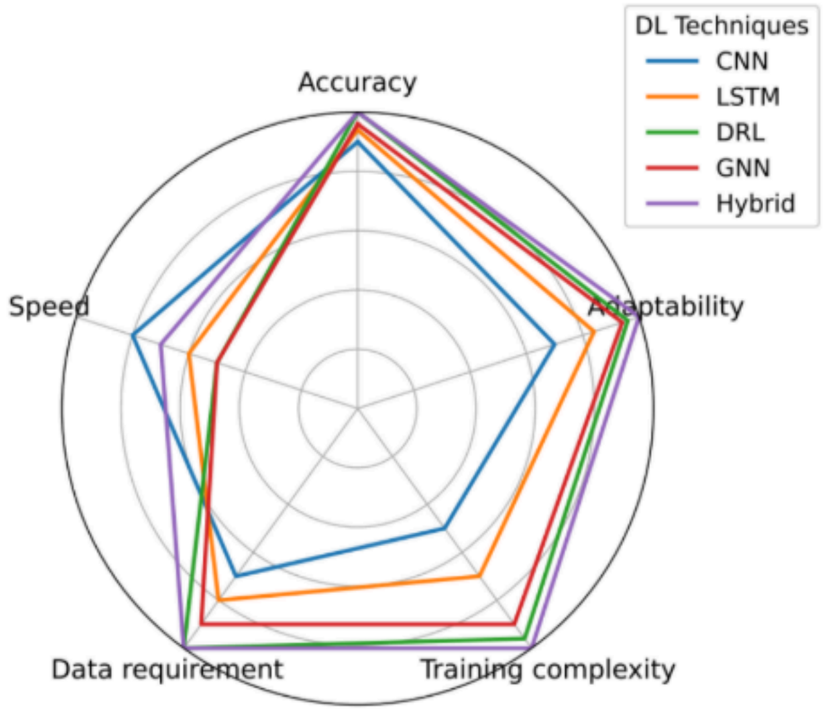
How do you know your data extraction is accurate?

- Answer: We used statistical metrics like Root-Mean-Square Erro (RMSE) and Cross-Correlation Analysis to prove our exporte waveforms are identical to the simulation source.
- Reference: See Data Extraction Reliability Research.

Why did you choose PSCAD or MATLAB for this?

- Answer: PSCAD is the gold standard for high-frequency transient MATLAB Engine API allows "zero-copy" transfer, moving data direct into Python memory for high-speed training.





How does the AI handle real-world noise?

- Answer: We deliberately trained our model on data with injected noise (AWGN) to make it resilient to real-world sensor inaccuracies and measurement jitter.

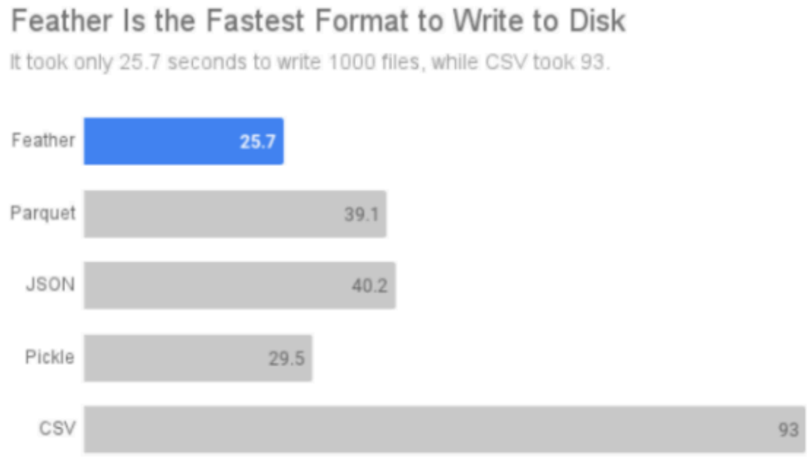
Why Parquet over CSV?

- Answer: CSV files bloat under high-frequency data and are slow to read. Parquet/Binary formats are columnar and highly compressed, making data loading significantly faster for the training pipeline.

Visual Reference: See benchmarks in Data Export Methods Research.

How will you meet the 10-20ms Tripping Deadline?

- Answer: We isolate and prune unnecessary layers (regression branch optimization) and will implement parameter quantization (converting 32-bit to 8-bit integers) to ensure the model footprint fits on microcontrollers.



https://towardsdatascience.com/best-file-format-to-store-large-data-dfa47701929f/